ARGUS: Identity Document Fraud Detection

Technical Report — FREUID Challenge 2026 (IJCAI-ECAI 2026)

Team: Team ARGUS
Author: Praveen Mittal
Repository: https://github.com/mittalpk/ARGUS

Commit SHA: 2c700286133d5a1297e4b6b5444ad1841320f969
Date: 2026-07-13

1. Introduction

Identity document fraud these days doesn't come from one place. Someone tampers with a physical card, someone else runs a genuine scan through a GenAI edit, and a third group just reprints a forged document and re-photographs it to wash out the digital artifacts. Three very different attack surfaces, three very different visual signatures. ARGUS started as an attempt to handle all three with a single model rather than building a separate detector per attack type, partly because that's what the competition asks for, and partly because in practice you don't get to choose which attack you're looking at before you look at it.

The project serves two purposes at once. For the competition it just needs to take a folder of test images and produce a **submission.csv** of fraud scores from inside a Docker container with no network access. But I built it with an eye toward an actual deployment too. There's audit logging, drift monitoring, and a human-review escalation path wrapped around the same classifier, because a document-fraud model that silently degrades in production is a much bigger problem than one that scores 0.6 instead of 0.7 on a leaderboard.

2. Method

2.1 Architecture

The codebase ("*src/models/baseline.py*", "*src/models/ensemble.py*") is built around three backbones:

Backbone	Input size	Why it's there
EfficientNet-B4	380×380	fast, cheap, the one we deployed
ConvNeXt-V2-Base	384×384	mid-size, better at texture and print artifacts
EVA-02-Large	448×448	biggest capacity, slowest, for when accuracy matters more than latency

All three share the same head design (ARGUSBackbone):

1. Take a *timm encoder*
2. Strip the classification head off
3. Bolt on Linear (features, 512)
4. GELU
5. Dropout
6. Linear (512, 1)

   Single logit is the **fraud score**. In ensemble mode the three logits get combined with learnable softmax weights, and each backbone's input is resized to whatever resolution it wants internally.

I want to be upfront about one thing: **the Docker image I'm submitting only ships EfficientNet-B4**, not the ensemble. That wasn't the plan originally but training all three to convergence on the full dataset wasn't something I could pull off inside the time and hardware I had for this submission (more on that in §5 and §6; it took some doing just to get one backbone trained on GPU).

I'm flagging EfficientNet-B4 early as the "if I can only ship one, ship this one" option, since it's the cheapest to run and easiest to train.

The ensemble code path works and is selectable with "***--model_name ensemble***". It just needs someone to train checkpoints for the other two backbones and drop them in.

2.2 Training

Nothing exotic here. AdamW, lr "**1e-4**", weight decay "**1e-2**", "**BCEWithLogitsLoss**" on the single logit. Augmentation is deliberately light — horizontal flip and a bit of brightness/contrast jitter, because document images have a fixed "up" orientation and I didn't want to teach the model to ignore that. Mixed precision ***("torch.cuda.amp")*** kicked in once training moved to GPU, mostly for memory headroom on an 8GB card rather than speed.

Seeding is one "***set_seed(42***)" call before anything gets constructed ***("Python random, NumPy, torch.manual_seed, cuda.manual_seed_all")***, and I also seed the DataLoader workers individually via "***worker_init_fn***". This one bit me early in development, since without it, changing "***num_workers***" quietly changed the augmentation sequence and made runs non-reproducible across machines.

Ten epochs, checkpointing whenever validation ***APCER @ 1% BPCER*** improved on the previous best. There's also a model-gate script ("***scripts/check_model_gate.py***") that compares a candidate run against the best historical run of the *same* architecture before letting it get promoted. I added the architecture filter after noticing the gate would otherwise happily compare an ***EfficientNet*** run against an old EVA-02 run's numbers, which isn't a fair comparison.

3. Data

The training set is the official **FREUID train_labels.csv** (69,352 labelled images), split (**40,005 genuine / 29,347 fraudulent**), across five document types (Benin, Egypt, Guinea, Mauritius, and Mozambique driving licenses/IDs), mixing physical and digital capture.

Ingestion (***"src/data/ingestion.py"***) opens every image, checks that it isn't corrupt, strips all EXIF/IPTC/XMP metadata, and re-saves it as a clean RGB JPEG (or RGBA PNG if it had real transparency). I also added path containment checks here after realizing the labels CSV is technically untrusted input. Nothing in this specific dataset tries anything adversarial with file paths, but there's no reason to trust that blindly just because it hasn't happened yet.

Splitting is a deterministic 70/15/15 stratified split, seed 42: 48,546 train / 10,403 validation / 10,403 test. No external data anywhere. The only outside input is the ImageNet-pretrained "***timm/efficientnet_b4.ra2_in1k***" backbone initialization, and that's only used during training. The inference container itself never touches the network; weights are baked in and **"pretrained=False"** is hardcoded at inference time.

4. Inference

The container reads a flat, read-only directory at "***/data***". Any of "***.jpeg /.jpg /.png /. webp /.bmp / .tif/ .tiff***", matched case-insensitively and writes one row per image to ***"/ submission/submission.csv"*** with columns **"id"** (filename minus extension) and **"label"** (sigmoid output in ***"[0, 1]"***, higher meaning more likely fraudulent). If an image is corrupt or unreadable, it gets a "**0.5"** default instead of crashing the whole run. One bad file shouldn't take down predictions for the rest of the batch.

Run command is the standard one from the reproducibility contract: "docker run -- ***rm --network none -v <test_dir>:/data:ro -v <out_dir>:/submissions <image>***". No network at inference; weights are already sitting in ***"/app/checkpoints/"*****.**

p95 latency measured "**16.4 ms"** per image on the GPU I trained on. On CPU, which is what the container falls back to if no compatible CUDA device is available. steady-state throughput measured at "**94.3 ms**" per image (737.4s for the full 7,821-image "**public_test**" batch on a 12-core CPU dev machine). The organizers have since confirmed reproducibility verification runs on a single A100 GPU, which torch==2.3.1 fully supports (unlike the sm_120 GPU used for local development, which falls back to CPU), so the judging environment is expected to use GPU inference, well within the 6-hour runtime requirement at either GPU or CPU speed.

5. Results

I want to slow down and be careful here, because the numbers look better than I trust them to be: zero, on both metrics, on both splits. For a competition explicitly framed

around "**next-generation**" fraud that's supposed to be *hard*, that's not a number I was going to write down without checking it first.

Metric	**Validation (10,403 images)**	**Held-out test (500-image sample)**
APCER @ 1% BPCER	0.0000	0.0000
AuDET	0.0000	0.0000
Accuracy	—	100.00%
p95 latency	16.4 ms (GPU)	—

So, I checked the two things that usually explain a suspiciously perfect score.

- **First, exact-duplicate images leaking across the train/val/test split:** I hashed 3,000 images from each side and compared them and found zero overlap.
- **Second, a metadata shortcut doing the model's job for it:** I checked whether ***"is_digital"*** or the document ***"type"*** field alone predicts the label, and neither does; both classes show up across every document type and both capture modes. Neither explanation held up.

I also ran the checkpoint against 50 real, unlabelled images from the actual public test set, just to see what it does on data it's never touched in any form. Every single prediction came back either under 0.05 or over 0.95, with nothing in between. That's not proof of anything by itself, but it's at least consistent with the model being genuinely this confident, rather than the split being somehow broken.

My honest read: This checkpoint has learned to separate genuine and fraudulent documents extremely well *for how this dataset was generated*. That's a known pattern with synthetic fraud/deepfake data: whatever pipeline produced the 'fraud' examples probably leaves a consistent fingerprint, and a CNN fine-tuned on enough examples will find it. Whether that fingerprint is the same thing the private test set is looking for, or whether the private set includes attack styles this training data doesn't have any examples of, I genuinely don't know. That's not something local validation can answer; only the actual leaderboard score can. So, take the table above as "**this is what I measured**," not as "**this is what to expect on the private set**."

6. Reproducibility

Runtime dependencies are pinned in **requirements.***txt (“torch==2.3.1”,” timm==1.0.3” , “albumentations==1.4.3”, rest in that file)*.

Worth flagging: the GPU I trained the shipped checkpoint on is an RTX 5070 Laptop GPU, which is newer than what ***“torch==2.3.1”*** ships kernels for (Blackwell, sm_120; the pinned build tops out at ***“sm_90”***). So, training happened in a separate, unpinned

environment (***"torch==2.11.0+cu128"***) purely to get GPU acceleration working, while everything else, including loading the resulting checkpoint back for inference, stayed on the pinned version.

A checkpoint is just a state_dict; I confirmed it loads and runs identically under **"torch==2.3.1"**, so this doesn't affect reproducibility of the actual submission, just how I trained it.

To reproduce from a clean clone:

1. **Environment:**
   "*bash scripts/setup.sh****"***
2. **Dataset** (needs Kaggle API credentials):
   "*bash scripts/download_data.sh****"***
3. **Train the champion checkpoint on the full dataset.** This took on the order of 10 minutes/epoch on GPU; expect it to be considerably slower on CPU only.
   *"PYTHONPATH=. python src/data/ingestion.py *
   *data/the-freuid-challenge-2026/train/train *
   *data/processed_full *
   data/the-freuid-challenge-2026/train_labels.csv"
   "PYTHONPATH=. python src/data/split.py data/processed_full data/splits_full"
   *"PYTHONPATH=. python src/training/train.py *
   *model=efficientnet training.epochs=10 training.seed=42 *
   *data.splits_dir=data/splits_full data.batch_size=16 *
   model.pretrained=true +model.use_amp=true"
4. **Promote the checkpoint, regenerate compliance evidence**
   "cp best_efficientnet_b4.pth checkpoints/champion_efficientnet_b4.pth"
   "python scripts/package_compliance_evidence.py"
5. **Build and run the actual submission container**
   "*docker build -t argus-freuid:local .*"
   "*docker run --rm --network none *
   *-v /path/to/flat/test/images:/data:ro *
   *-v "$(pwd)/out:/submissions" *
   argus-freuid:local"

Hardware: "*1× RTX 5070 Laptop GPU (8GB VRAM), 12-core CPU, 16GB RAM, WSL2"*. The shipped container itself doesn't need a GPU. It runs fine on CPU, just slower.

Two things went wrong during training that are worth mentioning in case someone else hits them:

- **CUDA** out-of-memory error at ***"batch_size=32"*** in full precision (dropped to ***"batch_size=16"*** with mixed precision, fixed it).

- **cuDNN** execution error that showed up partway through a run, which turned out to be a memory-fragmentation issue specific to this GPU's driver passthrough under WSL2.
  Setting ***"PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True"*** and running with **"num_workers=0"** made it go away. Neither of these is likely to show up on a standard cloud GPU box, but I'm noting them here because "it just crashed with no clear reason" cost me more time than it should have.